STAR BLASTER

TECHNICAL DESIGN DOCUMENT (TDD)

Engine Version: Unity 6 (6000.4.0f1)

Language: C# (.NET Standard)

Target Platforms: Mobile (Android/iOS), PC / WebGL

Architecture Pattern: Component-Based, Event-Driven, ScriptableObject-Driven, Object Pooling

1. System Architecture Overview

The architecture of **Star Blaster** is designed around modular, loosely coupled components to ensure high maintainability, testability, and performance optimization for mobile and desktop environments. Communication between disparate systems is handled primarily via a decoupled **Event-Driven Architecture (EventBus)** and **ScriptableObject configurations**.

[Input System] —> [PlayerController] —> [Shooter] —> [SharedProjectiles (Pool)]

|

[EnemySpawner] —> [PathFinding / WaveSO] —> [Enemy AI & Health]

2. Core Components & Scripts Technical Breakdown

A. Player & Input Management (PlayerController.cs)

- **Responsibility:** Captures input from Unity's modern **Input System** (supporting both touch screen drags on mobile and mouse cursor on desktop).
- **Boundary Clamping:** Dynamically calculates camera viewport boundaries using Camera.main.ViewportToWorldPoint() and applies Mathf.Clamp() alongside sprite dimension offsets (SpriteRenderer.bounds.size) to prevent the player ship from moving off-screen or encroaching on restricted UI zones.
- **Movement Interpolation:** Utilizes Vector3.MoveTowards() or direct transform updates scaled by Time.deltaTime for smooth drag response.

B. Weapon & Combat System (Shooter.cs, Projectile.cs, SharedProjectiles.cs)

- **Shooter (Shooter.cs):** Manages firing cadence for both Player and Enemies. Utilizes coroutines (FireContinuously) combined with real-time tracking (nextFireTime = Time.time + fireRate) to prevent fast-click spamming and ensure consistent attack intervals.
- **Projectile Pool (SharedProjectiles.cs):** Implements a centralized **Object Pooling pattern** to manage projectile instances. Instead of instantiating and destroying GameObjects at runtime (which causes garbage collection spikes), projectiles are pooled, activated, and deactivated on demand.

- **Projectile Behavior (Projectile.cs):** Handles individual projectile velocity (transform.up * speed), lifetime expiration (lifeTime), and self-deactivation back into the pool upon hitting targets or timing out.

C. Enemy Spawning & Movement (EnemySpawner.cs, WaveConfigSO.cs, PathFinding.cs)

- **Wave Configuration (WaveConfigSO.cs):** ScriptableObject assets storing wave data, including enemy prefabs, initial spawn counts, path references, and enemy-specific fire rates.
- **Pathfinding (PathFinding.cs):** Manages waypoint-based movement trajectories (linear paths) for incoming enemy formations.
- **Enemy Spawner (EnemySpawner.cs):** Iterates through active WaveConfigSO definitions, instantiating enemies to traverse the defined path sequences with randomized fire rate variances (fireRateVariance).

D. Health & Damage System (Health.cs, DamageDealer.cs)

- **Health (Health.cs):** Attached to any entity capable of taking damage (Player, Enemies). Manages current HP, triggers death events, publishes score/event metrics, and handles object destruction or pooling return.
- **Damage Dealer (DamageDealer.cs):** Component attached to projectiles and hazards. Detects trigger/collision events, extracts damage values, and passes them to the target's Health component before destroying or returning the projectile to the pool.

E. Game State, UI & Audio Management (UIManager.cs, ScoreManager.cs, AudioManager.cs, GameConfig.cs)

- **Game Configuration (GameConfig.cs / GameConfigSO.cs):** Centralized repository for global balancing parameters (base player speed, default fire rates, damage multipliers, difficulty curves).
- **Score Management (ScoreManager.cs):** Tracks current session scores, updates high score states, and broadcasts score change notifications to the UI layer.
- **UI Management (UIManager.cs, UIGameOver.cs):** Handles canvas state transitions (Main Menu, Gameplay HUD, Pause Overlay, Game Over Screen) and time scale freezing (Time.timeScale = 0 during pause states).
- **Audio Management (AudioManager.cs):** Centralized audio source controller for managing sound effects (SFX) and background music (BGM) playback volumes and clips.

F. Visual Polish & Effects (BackgroundScroller.cs, CameraShake.cs, VFXController.cs)

- **Background Scroller (BackgroundScroller.cs):** Implements UV texture offsetting or looping transform translation to create a seamless parallax starfield scrolling effect.
- **Camera Shake (CameraShake.cs):** Procedural trauma-based or coroutine-driven camera offset generator triggered upon taking damage or major explosions.

- **VFX Controller (VFXController.cs):** Manages particle systems for weapon impacts, engine exhausts, and destruction explosions.

3. Performance & Optimization Strategy

1. **Zero Runtime Allocation (Garbage Collection Minimization):**
   - Heavy reliance on SharedProjectiles Object Pooling for all bullets and recurring particle effects.
   - Avoidance of string concatenation inside Update() loops.
2. **Event-Driven Decoupling (EventBus.cs):**
   - Uses static C# event structures or generic pub/sub delegates to decouple subsystems (e.g., notifying score updates or power-up acquisitions without hard references between UI and gameplay logic).
3. **Modular ScriptableObjects:**
   - Game data (waves, stats, level configs) is entirely data-driven via ScriptableObjects (GameConfigSO, WaveConfigSO), reducing hardcoded values and facilitating rapid game balancing.